

Gradient Descent

Gradient Descent Intro:

$\Rightarrow$ If we already had OLS method to solve, then why we study gradient descent? OLS is mathematically excellent, but its closed form calculation can become computationally expensive for large/high dimensional datasets. In fact, the expensive part of calculating $\beta = (X^T X)^{-1} X^T Y$ is calculating $(X^T X)^{-1}$. That's why gradient descent.

$\Rightarrow$ What is Gradient Descent?

- Definition: It is an optimization technique used to find the minimum of a differentiable function.

- It's not limited to Linear Regression. It can be used whenever we have a differentiable objective/loss function that we want to minimize.

- Approach: Start with some values of the parameters and repeatedly adjust them in the direction that reduces the loss.

- Basic idea: Start $\rightarrow$ Calculate error

Calculate error $\leftarrow$ Move $\leftarrow$ Find direction $\leftarrow$ again
 $\rightarrow$ Move $\rightarrow$ Reach Minimum

- It's not a LR technique, it's an optimization technique.

$\Rightarrow$ Types of Gradient Descent:

- ① Batch Gradient Descent
- ② Stochastic Gradient Descent (SGD)
- ③ Mini-Batch Gradient Descent

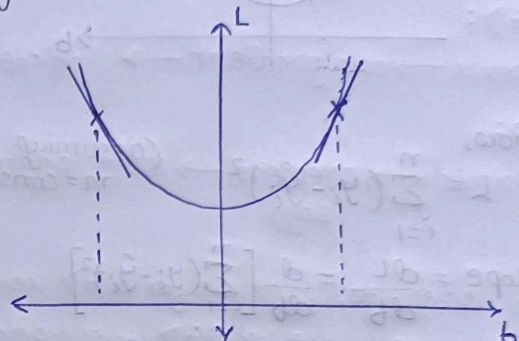
$\Rightarrow$ We know from OLS,

$$\hat{y}_i = mx_i + b$$

and,

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - mx_i - b)^2$$

Clearly, $L \propto b^2$, on graph of L vs b^2 : (approx)



Our basic aim is to decrease the value of L , we will either inc. b or dec. b , assuming we know, $m = \text{constant}$.

$\therefore$ if slope = -ve

$\therefore b$ shall inc.

if slope = +ve

$\therefore b$ shall dec.

For, this we use:

$$b_{\text{new}} = b_{\text{old}} - \text{slope}$$

But, directly subtracting slope can give drastic changes, so we use learning rate η there.

$$\therefore b_{\text{new}} = b_{\text{old}} - \eta \text{ slope}$$

where $\eta = \text{learning rate}$

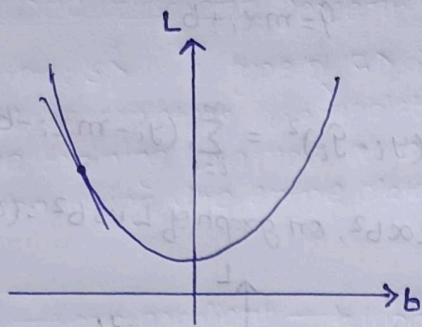
$= 0.01$ (generally,

can be changed too)

But when to stop finding the b_{new} ?

- ① when $b_{\text{new}} - b_{\text{old}} \approx 0.0001$ (very small)
- or,
- ② Limit the no. of iterations (or epochs)

⇒ Mathematical Formulation :



We know,

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (\text{assuming } m = \text{constant})$$

$$\therefore \text{slope} = \frac{dL}{db} = \frac{d}{db} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right]$$

$$\Rightarrow \frac{d}{db} \sum_{i=1}^n (y_i - mx_i - b)^2 = 2 \sum_{i=1}^n (y_i - mx_i - b)(-1)$$

$$\Rightarrow \frac{dL}{db} = -2 \sum_{i=1}^n (y_i - mx_i - b)$$

∴ Slope at $b=0$,

$$\left. \frac{dL}{db} \right|_{b=0} = -2 \sum_{i=1}^n (y_i - mx_i - 0)$$

$$\Rightarrow = -2 \sum_{i=1}^n (y_i - 78.35x_i)$$

(say, $m = 78.35$ is known as constant)

∴ b_{new} at $b=0$ or at b_{old} ,

$$b_{\text{new}} = b_{\text{old}} - \eta (\text{slope})_{b_{\text{old}}}$$

For each iterations (epoch) it will be calculated.

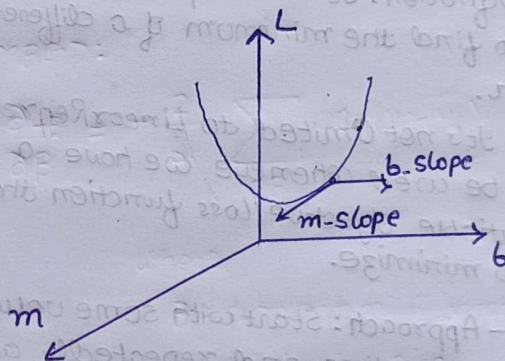
⇒ $\eta \times \text{slope}$ is called step size.

⇒ Btw, what is learning rate? It controls the size of each step we take while moving towards the min. of the loss function.

- Learning rate is too small → very slow
- " " " too " → missen minimum
- " " " appropriate → Best case
- Gradient = direction while Learning rate = step size

⇒ Always keep in mind that Gradient descent is an optimization technique whose basic requirement is differentiable functions.

⇒ Btw, in earlier derivation, we kept $m = \text{const}$, is it possible to calculate by not keeping any of m or b as constant? Yes!



$$\therefore L = \sum (y_i - \hat{y}_i)^2 = \sum (y_i - mx_i - b)^2$$

$$\therefore b\text{-slope} = \frac{\partial L}{\partial b} = -2 \sum (y_i - mx_i - b)$$

$$m\text{-slope} = \frac{\partial L}{\partial m} = -2 \sum (y_i - mx_i - b)x_i$$

Batch Gradient Descent :

⇒ But why do we have 3 types of gradient descent? The main reason is how much data we use to calculate the gradient before making one parameter update.

⇒ Suppose our dataset has 1000 rows.

Type	Data used for one update	Main characteristics
Batch GD	All 1000 rows	- Stable - accurate gradient dataset - Slow update for large
SGD	1 row	- Very fast update - noisy/unstable path
Mini-Batch GD	Small batch e.g., 32 rows	- good balance for speed & quality - efficient for large datasets

⇒ What is Batch GD? It is a GD algo. in which the gradient is calculated using the entire training dataset before updating the model parameters.

⇒ Note: Gradient Descent we studied in the intro is actually batch gradient descent.

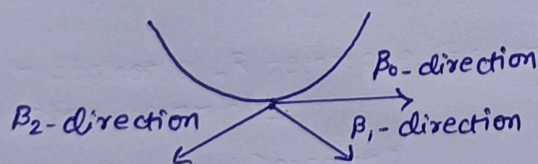
⇒ Since, earlier we studied with only one input, now we will go for n -input column. (say, start with 3 colⁿ, of which 2 be of input)

CGPA	ID	LPA

$X_1 \quad X_2 \quad Y$

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

So, we need to find $\{\beta_0, \beta_1, \beta_2\}$. To do so we will start with random values $\beta_0 = 0, \beta_1, \beta_2 = 1$, with epochs = 100 and learning rate = 0.1. So, $L(\beta_0, \beta_1, \beta_2)$ be in 4).



$$\beta_0 = \beta_0 - \eta \text{slope}$$

$$\beta_1 = \beta_1 - \eta \text{slope}$$

$$\beta_2 = \beta_2 - \eta \text{slope}$$

Now, we will start with $\frac{\partial L}{\partial \beta_0}$ for which L be as,

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where,

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

$$\hat{y}_1 = \beta_0 + \beta_1 X_{11} + \beta_2 X_{21}$$

$$\hat{y}_2 = \beta_0 + \beta_1 X_{21} + \beta_2 X_{22}$$

$$\therefore L = \frac{1}{2} \left[(y_1 - \beta_0 - \beta_1 X_{11} - \beta_2 X_{12})^2 + (y_2 - \beta_0 - \beta_1 X_{21} - \beta_2 X_{22})^2 \right]$$

$$\therefore \frac{\partial L}{\partial \beta_0} = \frac{1}{2} \left[2(y_1 - \hat{y}_1)(-1) + 2(y_2 - \hat{y}_2)(-1) \right]$$

$$\Rightarrow = -\frac{2}{n} \left[(y_1 - \hat{y}_1) + (y_2 - \hat{y}_2) + \dots + (y_n - \hat{y}_n) \right]$$

$$\Rightarrow = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\therefore \frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\text{Similarly, } \frac{\partial L}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{i1}$$

$$\frac{\partial L}{\partial \beta_2} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{i2}$$

$$\frac{\partial L}{\partial \beta_m} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) X_{im}$$

Stochastic Gradient Descent :

⇒ Definition: SGD is a type of GD in which the model parameters are updated using one training example at a time, rather than using the entire dataset.

⇒ BGD :

n -rows $\rightarrow$ calc. gradient $\rightarrow$ 1 update

SGD :

n -rows $\rightarrow$ 100 epochs $\rightarrow$ 100n updates (say)

⇒ For 100 epochs, n -rows, for BGD no. of updates = 100 while for SGD no. of updates = 100n.

⇒ SGD is generally faster per update and can reach a useful solution faster on very large datasets, but it does not necessarily mean SGD will always have lower total training time than BGD.

⇒ Why SGD is called 'Stochastic'? Bcz the gradient is calculated from one randomly selected/shuffled example, the gradient is a noisy estimate of the true gradient. That's why it's called 'stochastic GD'.

⇒ When to use it?

① Very large datasets

② When we have convex loss function

⇒ Learning Rate Schedule: It's a strategy for changing the learning rate during training, usually as the no. of epochs increase, we basically dec. the learning rate.

⇒ For SGD :

$$\frac{\partial L_i}{\partial \beta} = -2x_i(y_i - \hat{y}_i)$$

Mini Batch Gr.D. :

⇒ What it is? Type of GD where we divide the training dataset into small groups (mini-batches) and use one minibatch to calculate the gradient before updating the parameters.

⇒ MBGD :

32 $\rightarrow$ gradient $\rightarrow$ update

$$\frac{1000}{32} \approx 31.25 \therefore 32 \text{ updates per epoch.}$$

⇒ For MBGD :

$$\frac{\partial L_B}{\partial \beta} = -\frac{2}{b} \sum_{i \in B} x_i(y_i - \hat{y}_i)$$

Where,

B = current mini-batch

b = mini batch size

⇒ MBGD is a balance b/w BGD and SGD.

